Mentor - Session 2

Read the requirements from the book

VTS - Vacation Tracking System

Requirements

Vision

The main goal of this application is to improve the internal business processes of this organization, at least with respect to the time it takes to manage vacation time requests

Function - Req

* Rules-based system
* Enables manager approval (optional)
* E-mail notification [Manager - Employee]
* Uses the portal's single-sign-on mechanisms for all authentication
* Activity logs for all transactions [Auditing]
* HR and system administration override rules & [Auditing]
* Allows managers to directly award personal leave time
* Integration services

Non-Function - Req

* Uses existing hardware and middleware
* The system must be easy to use.[usability]

Constraints

* Integrations
* Legacy Hardware
* Single-sign-on
* User experience

Problem - Domain

Centered around the management of employee vacation time, sick leave, and personal time off within an organization.

Use - cases

Actors

* Managers
* Employees
* HR
* System Admin

Activities

* Managers
  - Approve Request
  - Award Time

* Employees
  - Manage Time
  [Create Requests]

* HR
  - Edit Employee Record
  - Manage Locations
  - Manage Leave Categories [Sick leave]
  - Override Leave Records

* System Admin
  - Backup System Logs

Deployment

Architecture

Existing Portal [as Mentioned]

Use-case [Manage Time]

Goal

Actor

Employee , Manager

Pre-Condition

Post-Condition

Alternative Flow

next use-case

Entities / Database Tables

Employee

- id
- name
- email
- emp_type_code
- manager_id
- created_on
- created_by

Employee type

type [manager , Emp , Hr , admin]

Q: Should i have two tables or one table?

Employee

Manager

Q: How many status here?

Vacation Type [Category]

Vacation Request

Vacation status

Employee Balance

Q:Should i have table for vacation type or not?

- type_id
- type_name
- type_code

- Request_id
- request_date
- vacation_status_id
- start_date
- end_date
- Employee_comment [description]
- Employee_id
- Manager_id ????
- vacation_type_id
- reject-comment
- balance

Q: Should I added it here or on the fly?

- status_id
- status_name
- status_code

Q:Should i have table for vacation status or not?

- Id
- employee_id
- type_id [Sick leave, Vacation leave, monthly leave]
- balance
- year
- current_balance

Read the use-case from the book

Employee and Manager Flow

Manager Flow

Manager Login

Select Req to approve or reject

Approve or Reject and update Request status

Notify Employee

DB

Employee Flow

Employee Login

Portal - Link to VTS [SSO]

Select vacation type & Fill data

API integration for Category Balance For Employee

Submit Request

Apply validations

valid

No

Display Errors

Yes

Save Request in DB

Notify Manager by Email

With Status Pending approval

Pseudocode

```
function void SaveRequest(request){
    boolean valid = validateRequest(request);
        if(valid){
                saveRequest(request);
                async notifyManager(request);
        } else{
           return errors
        }
    }
}
```

```
function boolean validateRequest(request){

        // validate date from , to
        // validate vacation category with balance
        validateRequestToAnddFrom(req.to , req.from);
        validateCategoryBalance(req.categoryId , employeeId , reqBalance);
}

function validateRequestToAnddFrom(req.to , req.from){

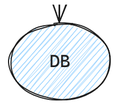
}

function validateCategoryBalance(req.categoryId , employeeId , reqBalance){

   Long empBalance = getEmployeeBalance(empId , categoryId)
    if(empBlanace > reqBalance)
            return true;

    return false;
}

getEmployeeBalance(empId , categoryId){
```



API integration HR legacy system
}

Top-down approach

* Start from Database design
  (High-level design)

Advantage:
* Provides a structured view of the database system from the start,
* Making it easier to understand the overall design.

Disadvantage:
* Changes at the top level can have a cascading effect on lower-level designs.

Bottom-up approach

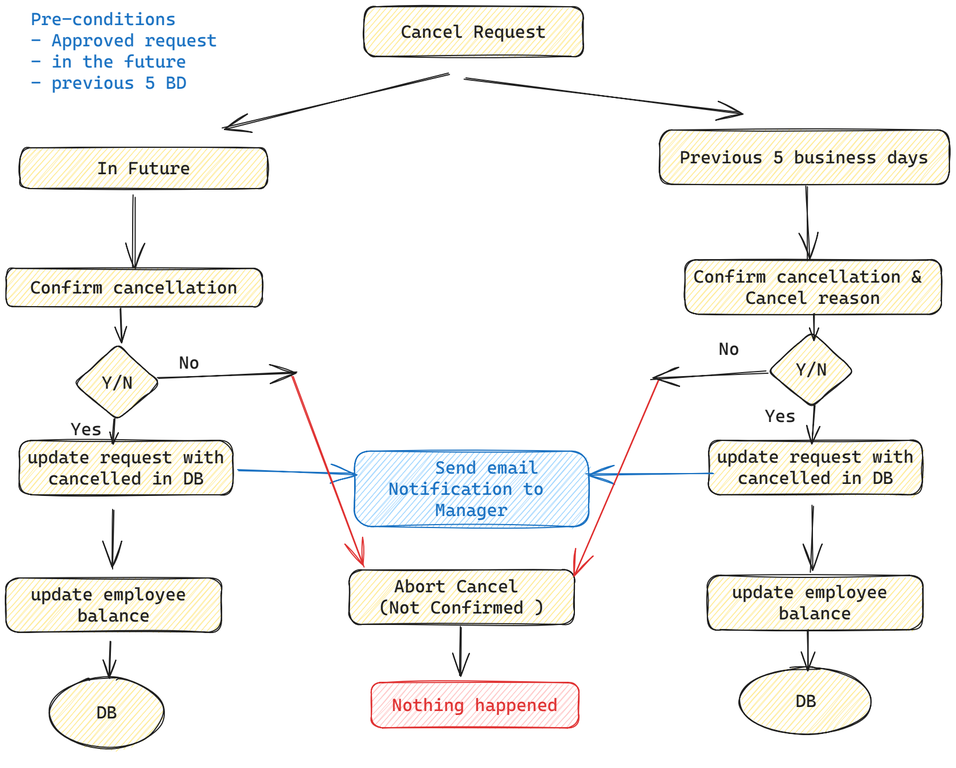
* Start by defining entities (Low level design)

Advantage:
* Allows for a more incremental approach
* More flexible in accommodating changes.
* Focuses on specific data needs

Disadvantage:
* It may lack an initial holistic view of the database system
* Can sometimes lead to a less optimized design

Cancel Request - use case

Pre-conditions
- Approved request
- in the future
- previous 5 BD




```
VTS/Requests/{id}/cancel
cancelRequest(requestDto){

    if(request in previous 5BD){
        if(cancel reason is empty)
          return error
    }

    service.cancelRequest(dto);
    {

        * cancelRequest(id, status);
        * updateEmployeeBalance(employeeId , 5);
        @async
        NotifyManager(employeeId , email text);
    }

}
```

Database tables changed ???

What's Next

* UML Course (Notation Diagrams) [YouTube Course]

* Sequence Diagram [YouTube video]
* Tools [Draw.io, plantuml]

Tasks

- Imagine the UI of Requests displayed to the Manager or Employee
- What if we need to have in the future another status like HR_Pending, HR_Approval
  with minimum change?
- Write the pseudo-code for the two cases above.
- Draw the state machine of the request.[ch5]
- Draw the sequence diagram of the request [ch5]
- Complete the rest of the use-case of this chapter [Cancel - Edit Pending Request]

Flowchart , sequence diagram - check database change

